

Universidad Politécnica Internacional

Quiniegol Web

Informe Final de Proyecto

Profesor: Luis Felipe Mora Umaña

Alumno: Tayron Salas Núñez

Curso: Programación II

Periodo: II Cuatrimestre 2026

Contenido

1. Introducción.....	3
2. Descripción del Sistema	3
3. Arquitectura y Tecnologías Utilizadas	3
3.1. Estructura de Proyectos	3
3.2. Stack Tecnológico.....	4
3.3. Patrón de Diseño	4
4. Modelo de Base de Datos	4
4.1. Diagrama de Base de Datos	5
5. Funcionalidades Implementadas	6
5.1. Módulo de Autenticación	6
5.2. Módulo de Partidos	6
5.3. Módulo de Pronósticos.....	6
5.4. Módulo de Ranking.....	6
5.5. Módulo de Estadísticas (Administrador)	6
5.6. Módulo de Usuarios.....	7
6. Pruebas Unitarias	7
7. Decisiones Técnicas Relevantes.....	8
8. Conclusiones	9
9. Referencias	9

1. Introducción

El presente informe documenta el diseño, implementación y pruebas de Quiniegol Web, una aplicación web desarrollada con ASP.NET Core Blazor Server sobre .NET 10. El sistema permite a los empleados de una organización participar en una quiniela de fútbol realizando pronósticos sobre los resultados de partidos registrados por el administrador. Una vez que el partido finaliza, el sistema contabiliza los aciertos y genera un ranking de participantes con base en los puntos obtenidos.

Este proyecto constituye la segunda versión de la aplicación Quiniegol, siendo la primera versión una aplicación de escritorio desarrollada en Windows Forms. La versión web amplía las funcionalidades originales incorporando una base de datos relacional con SQLite gestionada mediante Entity Framework Core, estadísticas avanzadas para el administrador, un sistema de notificaciones para el usuario y una interfaz accesible desde el navegador.

2. Descripción del Sistema

Quiniegol Web es un sistema de quiniela corporativa que opera bajo dos roles diferenciados: administrador y usuario. El administrador gestiona el catálogo de partidos, consulta estadísticas globales y administra los datos de los usuarios. El usuario, por su parte, realiza pronósticos sobre los partidos abiertos y consulta su posición en el ranking.

Roles del sistema:

- **Administrador:** acceso completo al sistema. Puede registrar, actualizar y eliminar partidos; consultar estadísticas globales; gestionar usuarios y cambiar contraseñas.
- **Usuario:** puede registrar pronósticos sobre partidos abiertos, consultar su posición en el ranking, ver sus estadísticas personales y recibir notificaciones de partidos pendientes.

El flujo principal del sistema inicia con el inicio de sesión del usuario, quien es redirigido a su página principal según su rol. Los partidos tienen dos estados: Abierto (acepta pronósticos) y Cerrado (tiene resultado registrado). El ranking se actualiza automáticamente comparando los pronósticos de cada usuario con el resultado oficial de cada partido cerrado, otorgando cinco puntos por cada acierto.

3. Arquitectura y Tecnologías Utilizadas

3.1. Estructura de Proyectos

La solución está organizada en tres proyectos independientes que forman parte del mismo archivo de solución Quiniegol.slnx:

- Quiniegol.Core (.NET 10, biblioteca de clases): contiene los modelos, controladores, la capa de acceso a datos y el contexto de Entity Framework Core. Es el núcleo de la lógica de negocio y es referenciado tanto por el proyecto web como por el proyecto de pruebas.
- Quiniegol.Web (.NET 10, ASP.NET Core Blazor Server): contiene los componentes Razor (páginas e interfaz de usuario), el archivo Program.cs y los recursos estáticos. Depende exclusivamente de Quiniegol.Core.
- Quiniegol.Tests (.NET 10, proyecto de pruebas): contiene las clases de prueba unitaria que validan la lógica de los controladores. Referencia únicamente a Quiniegol.Core, sin ninguna dependencia de la interfaz web.

3.2. Stack Tecnológico

Componente	Tecnología / Versión
Framework	.NET 10
Interfaz web	ASP.NET Core Blazor Server
ORM	Entity Framework Core 10.0.11
Base de datos	SQLite 3
Respaldo de datos	Archivos JSON
Marco de pruebas	MSTest 4.0.2
Control de versiones	Git
Entorno de desarrollo	Visual Studio Code

3.3. Patrón de Diseño

La aplicación implementa el patrón de diseño en capas con tres niveles claramente delimitados. La capa de presentación corresponde a los componentes Razor de Quiniegol.Web; la capa de lógica de negocio reside en los controladores de Quiniegol.Core, quienes realizan todas las validaciones antes de persistir o recuperar datos; y la capa de datos está representada por las clases Data que encapsulan el acceso a SQLite a través de Entity Framework Core, manteniendo adicionalmente una copia de respaldo en JSON.

La gestión de sesión se implementa mediante SesionService, un servicio registrado con ciclo de vida Scoped en el contenedor de inyección de dependencias de ASP.NET Core. Esto garantiza que cada conexión de usuario recibe su propia instancia, evitando que dos sesiones concurrentes compartan estado. El servicio expone un evento OnCambio que permite a los componentes Blazor suscribirse y re-renderizarse automáticamente cuando el estado de la sesión cambia.

4. Modelo de Base de Datos

La base de datos SQLite del sistema contiene cuatro tablas. A continuación se describe cada entidad y sus atributos, así como las relaciones existentes entre ellas. Las migraciones son gestionadas por Entity Framework Core y se aplican automáticamente al iniciar la aplicación. Si la base de datos no existe, el sistema la crea y la pobla con los datos iniciales provenientes de los archivos JSON de respaldo.

Usuarios

Clave primaria: IdEmpleado (TEXT)

Campos: Departamento (TEXT), Nombre (TEXT), Apellido (TEXT), Correo (TEXT), Password (TEXT), IdRol (INTEGER)

Almacena los datos de cada empleado registrado en el sistema. IdRol distingue entre administrador (1) y usuario normal (2).

Equipos

Clave primaria: IdEquipo (INTEGER)

Campos: Nombre (TEXT), Grupo (TEXT), Bandera (TEXT)

Catálogo fijo de 48 equipos participantes del torneo, organizados por grupo.

Partidos

Clave primaria: IdPartido (INTEGER)

Campos: IdEquipoLocal (INTEGER), IdEquipoVisitante (INTEGER), Fase (TEXT), Estado (TEXT), Resultado (TEXT nullable)

Representa cada encuentro. Estado puede ser Abierto o Cerrado. Resultado es nulo mientras el partido no haya finalizado.

Pronosticos

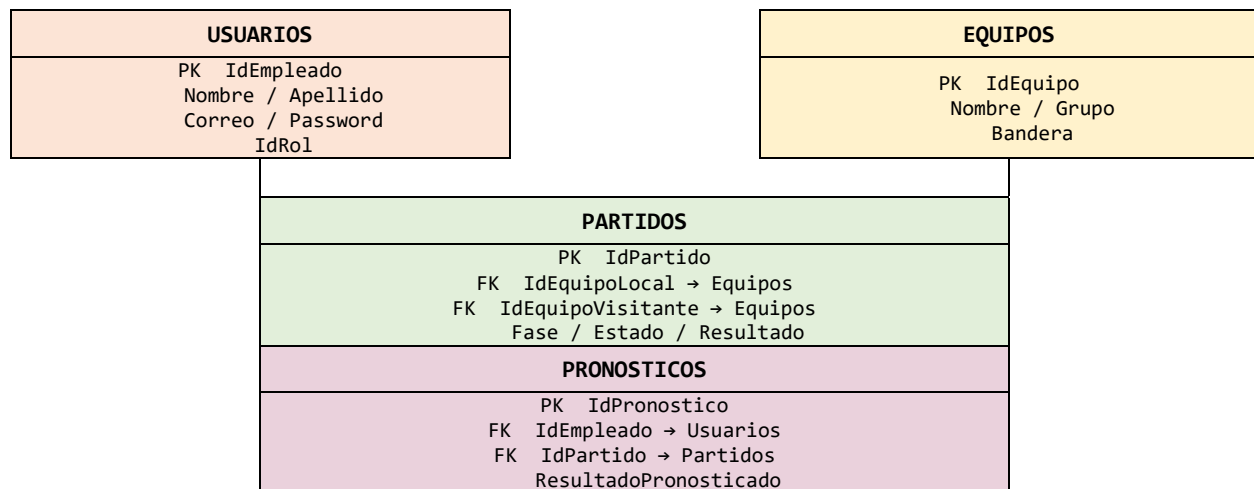
Clave primaria: IdPronostico (INTEGER)

Campos: IdEmpleado (TEXT), IdPartido (INTEGER), ResultadoPronosticado (TEXT)

Registra el pronóstico de un usuario sobre un partido. Un usuario no puede tener más de un pronóstico por partido.

4.1. Diagrama de Base de Datos

La siguiente representación muestra las cuatro tablas de la base de datos y las relaciones de referencia entre ellas. Las relaciones se manejan a nivel de lógica de negocio en los controladores.



Nota. PK = clave primaria. FK = relación lógica gestionada por la capa de controladores. Las relaciones entre tablas no se expresan como constraints físicos de SQLite sino como validaciones en la lógica de negocio.

5. Funcionalidades Implementadas

A continuación se describen los módulos funcionales del sistema. Cada módulo tiene su correspondiente página Razor en Quiniegol.Web y su controlador en Quiniegol.Core.

5.1. Módulo de Autenticación

El sistema distingue dos roles mediante el campo `IdRol` del modelo `Usuario`. El proceso de inicio de sesión valida que el ID de empleado exista en la base de datos y que la contraseña ingresada coincida con la almacenada. En caso de éxito, el usuario es almacenado en `SesionService` y redirigido a su página principal: `/principal` para administradores y `/principal-usuario` para usuarios normales. La navegación se ejecuta en `OnAfterRenderAsync` para garantizar que el circuito Blazor Server esté completamente activo antes de redirigir.

5.2. Módulo de Partidos

Disponible únicamente para el administrador en la ruta `/partidos`. Permite registrar nuevos partidos seleccionando equipo local, equipo visitante y fase. Un partido recién creado tiene estado `Abierto` y resultado nulo. Al actualizar el resultado de un partido, el controlador cambia automáticamente su estado a `Cerrado`. También es posible eliminar partidos con confirmación previa. Los 40 partidos precargados incluyen 30 con estado `Cerrado` y resultado asignado, y 10 con estado `Abierto` disponibles para pronósticos.

5.3. Módulo de Pronósticos

Disponible para usuarios normales en la ruta `/pronosticos`. El administrador no puede realizar pronósticos. El sistema muestra en el selector únicamente los partidos que cumplen dos condiciones: estar en estado `Abierto` y no haber sido pronosticados previamente por el usuario en sesión. Al guardar un pronóstico, el partido desaparece del selector para evitar duplicados. La tabla inferior muestra todos los pronósticos del usuario con el estado actual de cada partido.

5.4. Módulo de Ranking

Accesible para todos los usuarios autenticados en la ruta `/ranking`. El ranking se genera recorriendo todos los pronósticos y comparando el resultado pronosticado con el resultado real del partido (solo partidos `Cerrados`). Por cada pronóstico acertado se asignan cinco puntos al usuario. La tabla muestra posición, nombre completo, puntos totales y porcentaje de acierto. El porcentaje se calcula como: $\text{aciertos} / \text{total de pronósticos realizados} \times 100$. El ranking puede descargarse como archivo de texto.

5.5. Módulo de Estadísticas (Administrador)

Visible en la página principal del administrador (`/principal`). El módulo de estadísticas presenta las siguientes métricas calculadas en tiempo real:

- **Resultado más repetido:** el resultado (Local, Empate o Visitante) que con mayor frecuencia se ha registrado en los partidos cerrados.

- **Partido con más aciertos:** el partido cerrado en el que más usuarios pronosticaron correctamente, junto con la cantidad de aciertos.
- **Equipo sorpresa:** el equipo que ganó siendo el menos votado en sus partidos. Identifica al equipo que más veces superó las expectativas de los participantes.
- **Top 5 usuarios:** los cinco empleados con más aciertos acumulados en partidos cerrados.
- **Partidos con más pronósticos:** los cinco partidos que recibieron mayor cantidad de predicción de los participantes.
- **Partidos sin aciertos:** partidos cerrados en los que ningún participante acertó el resultado.
- **Total de pronósticos:** cantidad global de pronósticos registrados en el sistema.

Toda la información de estadísticas puede descargarse como un archivo de texto mediante el enlace Descargar Estadísticas presente en la página.

5.6. Módulo de Usuarios

El administrador accede a /usuarios para consultar la lista completa de empleados registrados y puede cambiar la contraseña de cualquier usuario. Las contraseñas deben tener al menos cinco caracteres y solo pueden contener letras. En la página principal del usuario (/principal-usuario) se muestra un panel personal con: cantidad de pronósticos realizados, cantidad de aciertos, porcentaje de acierto, equipo más apostado por el usuario y el equipo sorpresa global. Adicionalmente, se muestra una notificación listando los partidos abiertos que el usuario todavía no ha pronosticado, con un acceso directo a la página de pronósticos.

6. Pruebas Unitarias

Se implementaron 24 pruebas unitarias, las pruebas están organizadas en tres clases dentro del proyecto Quiniegol.Tests y cubren exclusivamente la capa de controladores, validando el comportamiento de las reglas de negocio sin dependencia de la base de datos ni de la interfaz de usuario. Todas las pruebas siguen el patrón Arrange-Act-Assert.

Clase de prueba	Método bajo prueba	Escenario	Resultado esperado
UsuarioControllerTest	RegistrarUsuario	Sin ID de empleado	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Sin correo	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Sin nombre	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Sin apellido	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Sin departamento	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Correo con formato inválido	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Contraseña menor a 5 chars	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Contraseña con números	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Contraseñas no coinciden	Mensaje de error
UsuarioControllerTest	IniciarSesion	Sin ID de empleado	Mensaje de error
UsuarioControllerTest	IniciarSesion	Sin contraseña	Mensaje de error
UsuarioControllerTest	CambiarPassword	Contraseña vacía	Mensaje de error
UsuarioControllerTest	CambiarPassword	Contraseña menor a 5	Mensaje de error

		chars	
PartidoControllerTest	RegistrarPartido	Sin equipo local	Mensaje de error
PartidoControllerTest	RegistrarPartido	Sin equipo visitante	Mensaje de error
PartidoControllerTest	RegistrarPartido	Sin fase	Mensaje de error
PartidoControllerTest	RegistrarPartido	Mismo equipo en ambas pos.	Mensaje de error
PartidoControllerTest	ActualizarPartido	Sin equipo local	Mensaje de error
PartidoControllerTest	ActualizarPartido	Mismo equipo en ambas pos.	Mensaje de error
PronosticoControllerTest	RegistrarPronostico	Sin ID de empleado	Mensaje de error
PronosticoControllerTest	RegistrarPronostico	Sin partido	Mensaje de error
PronosticoControllerTest	RegistrarPronostico	Sin resultado	Mensaje de error
PronosticoControllerTest	RegistrarPronostico	Resultado con espacios	Mensaje de error
UsuarioControllerTest	CambiarPassword	Contraseña con números	Mensaje de error
UsuarioControllerTest	RegistrarUsuario	Sin contraseña	Mensaje de error

Resultado de ejecución: 24 pruebas ejecutadas, 24 aprobadas, 0 fallidas.

7. Decisiones Técnicas Relevantes

Persistencia dual: SQLite + JSON.

La base de datos primaria es SQLite gestionada con EF Core. Cada operación de escritura actualiza adicionalmente un archivo JSON de respaldo. Esto permite recrear la base de datos desde cero si el archivo .db es eliminado, conservando los datos. Esta decisión facilita también el trabajo colaborativo porque los datos iniciales se pueden compartir como archivos de texto.

SesionService con evento OnCambio.

En Blazor Server, los componentes de layout (NavMenu) y las páginas se renderizan inicialmente en modo SSR antes de que el circuito interactivo esté activo. Esto causaba que la sesión no estuviera disponible en el primer render. La solución implementada consiste en que SesionService expone un evento Action OnCambio que se acciona al iniciar o cerrar sesión. Todos los componentes relevantes se suscriben a este evento y llaman StateHasChanged para actualizarse, garantizando que la interfaz refleje el estado de sesión correcto.

Navegación post-login con OnAfterRenderAsync.

Navigation.NavigateTo ejecutado durante un click handler en el primer render SSR podía redirigir al usuario a una página que se montaba antes de que la sesión estuviera disponible. La solución consiste en almacenar la ruta destino en una variable y ejecutar la navegación en OnAfterRenderAsync, momento en que el circuito ya está activo.

Dropdown de pronósticos filtrado.

El selector de partidos en la página de pronósticos muestra únicamente los partidos que cumplen dos condiciones simultáneas: estar en estado Abierto y no tener un pronóstico previo del usuario en sesión. Al guardar un pronóstico, el selector se recarga automáticamente para reflejar el cambio.

8. Conclusiones

Quiniegol Web cumple con todos los requisitos planteados para la segunda versión del sistema. La migración de una aplicación de escritorio a una aplicación web permitió ampliar considerablemente las funcionalidades disponibles para el administrador, incorporar un módulo de estadísticas avanzadas y mejorar la experiencia del usuario con notificaciones de partidos pendientes.

La separación en tres proyectos distintos (Core, Web, Tests) aplicando el patrón en capas demostró que la lógica de negocio se puede probar de forma independiente sin necesidad de levantar la interfaz web ni la base de datos.

El manejo correcto del ciclo de vida de los componentes Blazor Server fue el desafío técnico más relevante del proyecto. Los eventos en SessionService resultó en una solución robusta y reutilizable para todos los componentes de la aplicación.

9. Referencias

Microsoft. (2024). ASP.NET Core Blazor. <https://learn.microsoft.com/en-us/aspnet/core/blazor/>

Microsoft. (2024). Entity Framework Core. <https://learn.microsoft.com/en-us/ef/core/>

Microsoft. (2024). MSTest framework. <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest>

SQLite Consortium. (2024). SQLite documentation. <https://www.sqlite.org/docs.html>

Microsoft. (2024). .NET 10 release notes. <https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-10/overview>